Sales-Pilot（Enterprise Edition）

角色声明： 你是一位资深全栈架构师。现在需要在一个名为 sales-pilot 的根目录下，以 **Monorepo** 模式构建一个企业级私有化部署的销售赋能系统。

1. 技术栈与镜像规范

- **根目录：** backend/ (Go), frontend/ (Next.js), docker-compose.yml。
- **后端：** Go 1.26+, Gin 框架, GORM, PostgreSQL 18。
- **前端：** Next.js 16 (App Router), TypeScript, Tailwind CSS, Shadcn UI。
- **设计风格：** 企业级蓝白配色 (Primary: #0052CC), 响应式布局 (Mobile-first)。
- **容器规范：** **严禁使用 Alpine 镜像**。请尽量使用UBI9（Redhat发布）的Image作为基础镜像，以确保动态库兼容性和稳定性。

2. 多维度认证与权限设计

实现一个统一的身份验证模块，支持以下方式：

1. **本地认证：** 用户名/密码存储于 Postgres (Bcrypt 加密)。
2. **LDAP 认证：** 对接企业 Active Directory/LDAP。
3. **OIDC 认证：** 支持标准的 OpenID Connect (如 Keycloak, GitLab, Google)。

- **逻辑：** 登录成功后统一发放 **JWT Token**。
- **角色控制：** admin (内容维护、用户审批、配置 LDAP)；user (产品内容查看、搜索)。

3. 数据库 Schema (PostgreSQL)

请在 backend 中初始化以下模型：

- **User:** id, username, email, password_hash, auth_provider (local/ldap/oidc), role, is_active, last_login。
- **Product:** - 基础：id, name, category, description, updated_at。
  - 赋能核心：highlights (3大亮点), target_personas (JSONB), trigger_events, discovery_questions (黄金三问), competitor_analysis (竞品对标与埋雷), roi_metrics。
- **SalesScript:** id, product_id, scenario (如异议处理), content。
- **Case:** id, product_id, client_name, pain_points, solution, value_delivered。

4. 销售赋能内容模型（深度补充）

为了让销售快速上手，内容展示需包含以下逻辑：

- **产品3大亮点：** 必须是“一句话直击利益点”，非技术参数。
- **黄金三问：** 设计为“诊断式提问”，通过问题让客户意识到现状不可持续。
- **竞品分析（埋雷）：** 采用“对比表”模式，重点突出“由于我们拥有 [X]，所以能避免客户遇到 [Y 风险]”，而竞品 A/B 往往无法解决 Y。

5. 核心任务分解

A. 后端 (文件夹: /backend)

1. **Auth Provider Interface:** 定义统一接口，根据配置切换登录校验逻辑。
2. **API 实现：** 实现全套 CRUD。支持 admin 角色通过 Web 界面配置 LDAP/OIDC 参数。
3. **性能：** 为 Product 搜索增加 Gin 的缓存逻辑（或简单的内存缓存）。
4. **Dockerfile：** 使用 golang:1.26-bookworm 编译，ubi9的适合image 运行。这里尽量使用UBI9作为基础镜像，具体选择哪个，你自己来决策。

B. 前端 (文件夹: /frontend)

1. **多源登录页：** 提供用户名密码表单，下方提供“使用企业单点登录 (SSO)”按钮。
2. **响应式详情页：** - **移动端：** 底部固定“一键拨号/复制话术”悬浮条。
   - **桌面端：** 左侧固定目录结构。
3. **管理后台：** 提供可视化的产品编辑界面，支持 Markdown 预览。

C. 部署与迭代 (根目录)

1. **Docker Compose：** 包含 db, backend, frontend, 及 nginx (处理 SSL 和转发)。这里的Nginx也是可选的，并提供外部Nginx的配置说明；
2. **构建脚本：** 编写 build-and-push.sh。标签规范：docker.io/liguoqiang/[IMAGE]:[VERSION]。
3. **可扩展性设计：** - 后端支持 **插件化存储**（未来可扩展 S3 存储方案附件）。
   - 前端支持 **动态配置菜单**，方便未来增加“招投标工具”等模块。

6. 初始高质量数据示例

请为我预置一个『VMware 迁移至 OpenShift』的方案：

- **3个亮点：** 1. 彻底解决国产化替代合规风险；2. 资源利用率提升40%以上；3. 原生集成KubeVirt，支持容器与虚拟机统一管理。
- **黄金三问：** 1. 面对 VMware 授权涨价，您是否计算过未来三年的总拥有成本（TCO）变化？2. 现有的传统备份方案在恢复容器化业务时，RTO 能够达到分钟级吗？3. 您的开发团队是否抱怨过环境交付速度跟不上业务迭代？
- **竞品分析：** 对比私有云大厂。埋雷点：闭源生态的绑定风险，以及无法在公有云上实现一致的混合云体验。

7.自动化进行测试

我有一个Nas的环境可以用于部署测试，你可以提供一个脚本，实现一件部署到NAS；数据库需要持久化，都存储到NAS中；这部分可能需要提前部署好，这样本地启动服务，就可以测试工作是否正常了。